

How OS Creates a Process

1. What is a Program?

A **program** is a file that contains **compiled code** (machine instructions + static data). It sits in storage (HDD/SSD) and **does nothing** until the OS runs it.

Example:

a.out, chrome.exe, python.exe — all are programs stored on disk.

2. What is a Process?

A **process** is a **program in execution**.

The moment the OS starts running a program, it becomes a process.

A process =

- Program code in memory
- CPU registers
- Stack
- Heap
- PCB (metadata)
- Open files
- Execution state

So a process is a *living, running entity*, whereas a program is just *dead code lying on disk*.

3. How does the OS create a Process?

To convert a program into a process, OS performs a series of steps.

This is called **Process Creation**.

Step-by-Step Explanation

Step 1: Load the Program from Disk to Main Memory

The OS loads:

- **Text segment** → machine code
- **Initialized static/global data**
- **Uninitialized data (BSS)** → set to zero

This forms the basic memory layout of the process.

Example:

If you run `./a.out`, OS copies `a.out`'s machine code into RAM.

Step 2: Allocate Runtime Stack

OS creates a **stack** for the process.

The stack stores:

- Function call frames
- Local variables
- Return addresses
- CPU registers during context switch

Stack grows downward (traditionally).

Every process gets its own stack.

Step 3: Allocate Heap

Heap is used for **dynamic memory** (`malloc`, `new`).

OS reserves space for the heap area; it grows upward as needed.

Stack ↘

Heap ↗ (they grow toward each other in classical systems)

Step 4: Allocate and Set Up PCB (Process Control Block)

OS creates a **PCB** to manage the process.

PCB contains:

- PID (process ID)
- Program counter
- CPU registers
- Scheduling info
- Memory boundaries
- Open file table

The PCB is the “identity” of a process in OS.

Step 5: Set Up I/O Resources

OS handles I/O connections like:

- Standard input
- Standard output
- Standard error

Example:

Process's output goes to terminal → OS manages it.

If the process needs files/network connections, OS tracks them in PCB.

Step 6: Set Initial CPU State and Transfer Control to main()

OS sets:

- Program Counter (PC) → address of main()
- Stack pointer → top of the stack
- Registers → initial values

Then OS **schedules** the process on CPU.

Finally, CPU jumps to **main()**, and the program starts running.

4. Architecture of a Process

A process's memory is divided into **four classical sections**.

This layout has remained the same in operating system design for decades.

1. Text Segment

- Contains **compiled machine code** loaded from the program file.
- Read-only to avoid accidental overwrites.

Example:

main(), loops, instructions — all live here.

2. Data Segment

Contains all **global** and **static** variables.

Two parts:

- **Initialized data** (global = 10)
- **Uninitialized data (BSS)** (static int x;)

These are loaded during process creation.

3. Heap

- Used for **dynamic memory** (malloc, new).
- Grows **upward** as the program requests more memory.
- Managed at runtime.

Example:

```
int* arr = (int*) malloc(100 * sizeof(int));
```

4. Stack

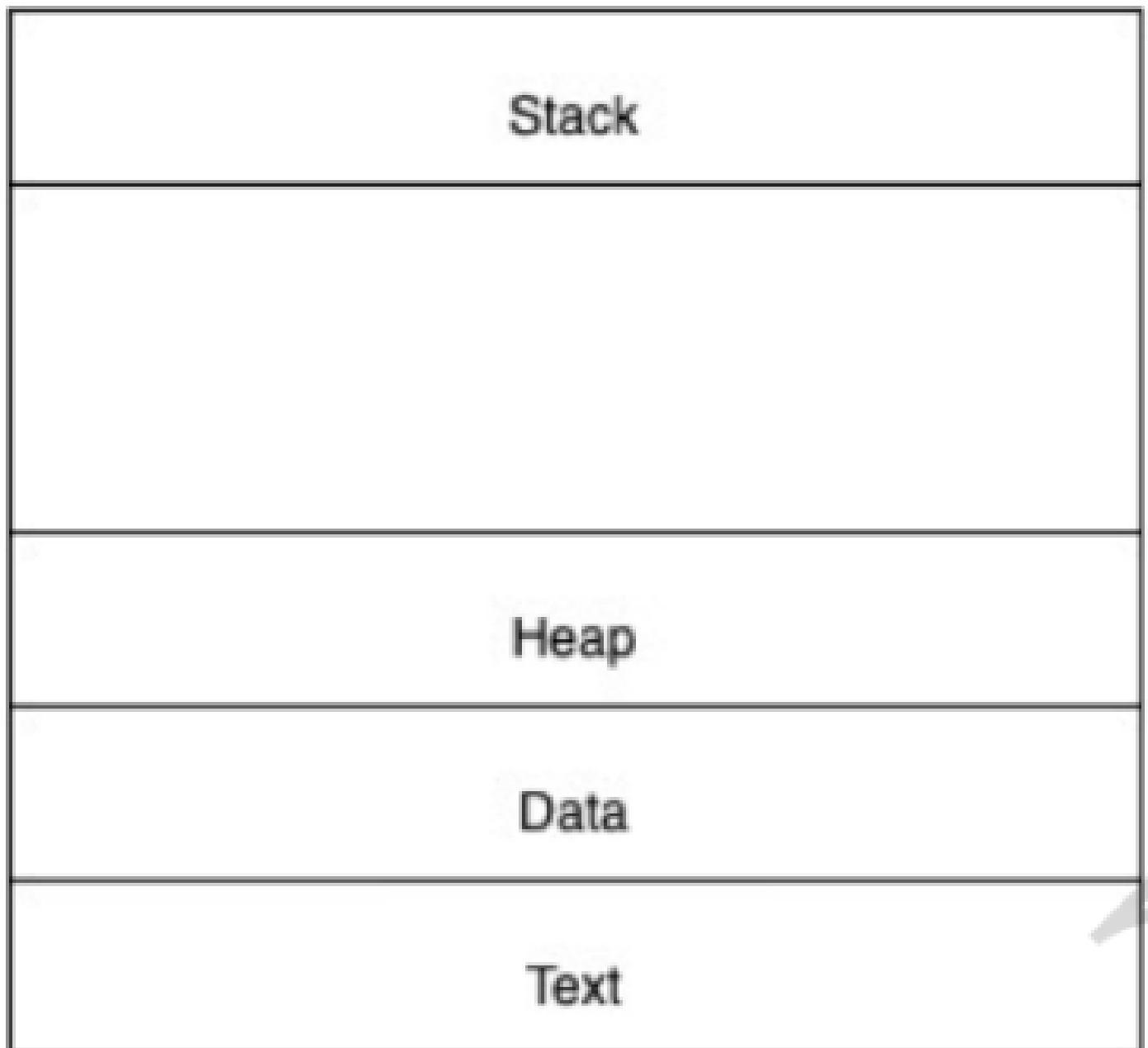
- Stores **local variables, function arguments, return addresses**.
- Grows **downward**.
- Every function call creates a *stack frame*.

Example:

Inside a function:

```
int x = 5;
```

This lives on the stack.



5. Attributes of a Process

To manage processes, the OS needs to identify and track them.
This is done through:

(a) Feature to Uniquely Identify

Every process has a **PID (Process ID)**.

Example:

Linux command `ps -aux` displays all PIDs.

(b) Process Table

- A **table-like data structure** maintained by OS.
- Contains entries for *every active process*.
- Each entry is a **PCB (Process Control Block)**.

Think of the Process Table as a **database** of all running processes.

(c) PCB (Process Control Block)

PCB stores **all important information** about a process.

Why?

Because the OS must pause and resume processes, and it cannot afford to forget their state.

The PCB stores:

- Process ID
- Program counter
- Process state
- Priority
- CPU registers
- List of open files
- List of allocated devices

In short:

PCB = Process Identity + Process State + CPU Context + Resources

6. PCB Structure (Detailed)

The image shows the classical PCB layout:

1. Process ID

- Unique identifier assigned by OS.

2. Program Counter

- Stores the **address of the next instruction** to execute.

3. Process State

Possible states:

- New
- Ready
- Running
- Waiting
- Terminated

OS updates this field constantly.

4. Priority

- Determines which process gets CPU first.
- Higher priority → more CPU time.

5. Registers

- General-purpose CPU register values.
- Stored when process is paused.
- Restored when process is scheduled again.

6. List of Open Files

- All files opened using `fopen()`, `read()`, `write()`, etc.

7. List of Devices

- Any device assigned to the process (keyboard, display, network).

Process ID	Unique identifier
Program Counter (PC)	Next instruction address of the program
Process State	Stores process state
Priority	Based on priority a process gets CPU time
Registers	
List of open files	
List of open devices	

7.Registers in the PCB

When a process is running:

- It uses **CPU registers** to store temporary values.

But when its **time slice expires**, CPU switches to another process.

To resume later, OS must remember the previous CPU state.

So OS **stores all CPU register values inside the PCB**.

This is called **context switching**.

Why this is important?

Because when the process runs again:

- OS reads those register values from PCB
- Writes them back into CPU registers
- The process continues **from the exact point it paused**

This is how **multitasking** works smoothly.

InShort

Process Architecture

- Text → code
- Data → global/static
- Heap → dynamic memory
- Stack → function calls

PCB

- Represents a process in the OS
- Stores PID, PC, registers, state, priority, files

Why registers are stored?

To pause and resume processes without losing progress.